

為 Dialogflow 應用程式建立前端 Django 用戶端

來源：<https://codelabs.developers.google.com/codelabs/chatbots-dialogflow-django-frontend?hl=zh-tw>

步驟數：14

瞭解如何建立前端 Ruby 用戶端，為 Dialogflow 應用程式建立對話體驗。

目錄

1. [1. 事前準備](#)
2. [2. 架構總覽](#)
3. [3. 下載並執行前端應用程式](#)
4. [4. 設定本機環境](#)
5. [5. 服務帳戶設定](#)
6. [6. 設定要從應用程式呼叫的 Dialogflow detectIntent 端點](#)
7. [7. 在本機建構及執行應用程式](#)
8. [8. 將應用程式部署至 App Engine 標準環境](#)
9. [9. 測試前端用戶端](#)
10. [10. 正式版](#)
11. [11. 準備好在實際工作環境提供內容時，請在 mysite/settings.py 中將 DEBUG 變數變更為 False。](#)
12. [12. 測試您的聊天機器人](#)
13. [13. 清除所用資源](#)
14. [14. 恭喜](#)

1. 事前準備

在本程式碼研究室中，您將學習如何建構前端 Django 用戶端，為 Dialogflow 應用程式建立對話式體驗。具體來說，您將執行下列操作：

- 下載、設定及執行 Django 前端用戶端。
- 設定要從 Django 前端用戶端呼叫的 Dialogflow detectIntent 端點。
- 將應用程式部署至 Google Cloud 上的 App Engine。
- 測試日曆邀請是否會根據使用者要求設定。

必要條件

繼續操作前，請先完成下列程式碼研究室：

1. [使用 Dialogflow 建構預約排程器](#)
2. [瞭解 Dialogflow 中的實體](#)
3. [瞭解如何整合 Dialogflow 與 Google 日曆，完成預約程序](#)

課程內容

- 如何設定及執行 Dialogflow 的 Django 前端用戶端
- 如何將 Django 前端用戶端部署至 Google Cloud App Engine
- 如何透過自訂前端測試 Dialogflow 應用程式

建構項目

- 您將設定及執行 Dialogflow 的 Django 前端用戶端。
- 您會將 Django 前端用戶端部署至 Google Cloud 的 App Engine。
- 您將從該自訂前端測試 Dialogflow 應用程式。

軟硬體需求

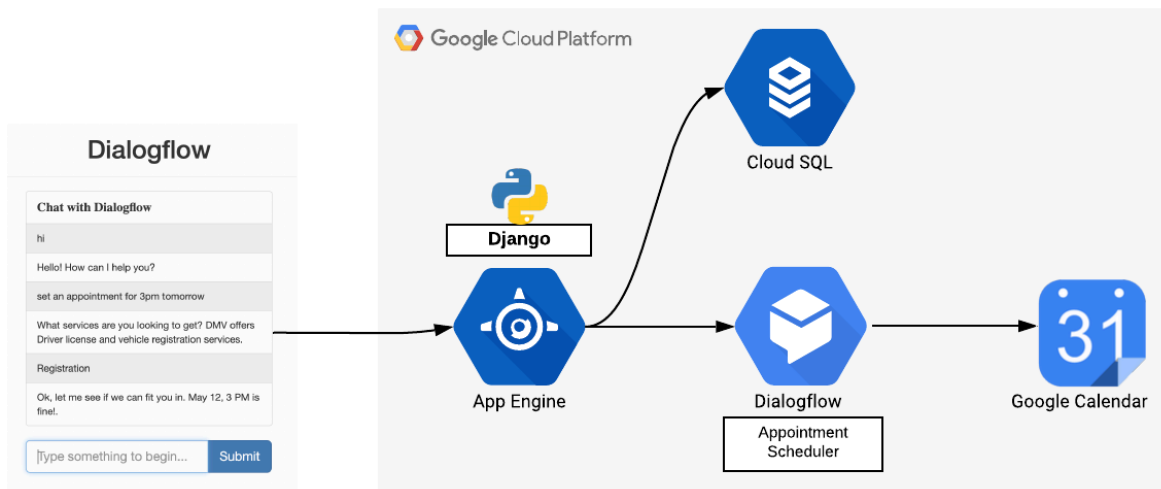
- 對 Python 有基本瞭解
 - 對 Dialogflow 有基本瞭解
-

步驟 2 · 約 3 分鐘

2. 架構總覽

您將使用先前建構的「預約排程」對話體驗，為應用程式建立自訂前端。您將使用 Django 建構前端、在本機執行及測試，然後部署到 App Engine。

使用者會透過前端傳送預約要求，這項要求會呼叫 Dialogflow detectIntent API，以在要求的日期和時間預約。接著，Dialogflow 執行要求會傳送要求至 Google 日曆，設定相應的預約時間，並透過 Dialogflow 將確認訊息傳回給使用者。



最終結果如下所示：

Dialogflow

Chat with Dialogflow	
hi	
Hello! How can I help you?	
set an appointment for 3pm tomorrow	
What services are you looking to get? DMV offers Driver license and vehicle registration services.	
Registration	
Ok, let me see if we can fit you in. May 12, 3 PM is fine!.	

步驟 3 · 約 2 分鐘

3. 下載並執行前端應用程式

1. 在電腦的本機終端機中輸入下列指令，將存放區複製到本機電腦：

```
git clone https://github.com/priyankavergadia/Django-Dialogflow-Appointment-Scheduler.git
```

2. 切換至包含程式碼的目錄。您也可以下載 zip 格式的範例，然後解壓縮該檔案。

```
cd Django-Dialogflow-Appointment-Scheduler
```

步驟 4 · 約 3 分鐘

4. 設定本機環境

部署作業完成後，應用程式會使用 App Engine 標準環境內建的 Cloud SQL Proxy 與您的 Cloud SQL 執行個體進行通訊。不過，如要在本機測試應用程式，您就必須在開發環境安裝並使用 Cloud SQL Proxy 本機副本。詳情請參閱「[關於 Cloud SQL Proxy](#)」。

如要在 Cloud SQL 執行個體上執行基本的管理工作，您可以使用 MySQL 適用的 Cloud SQL 用戶端。

注意：您必須驗證 gcloud 指令列工具，才能從本機電腦使用 Proxy 連線。

安裝 Cloud SQL Proxy

下載並安裝 Cloud SQL Proxy。Cloud SQL Proxy 用於在本機執行時連線至 Cloud SQL 執行個體。

下載 Proxy。

```
curl -o cloud_sql_proxy https://dl.google.com/cloudsql/cloud_sql_proxy.darwin.amd64
```

將 Proxy 設為可執行。

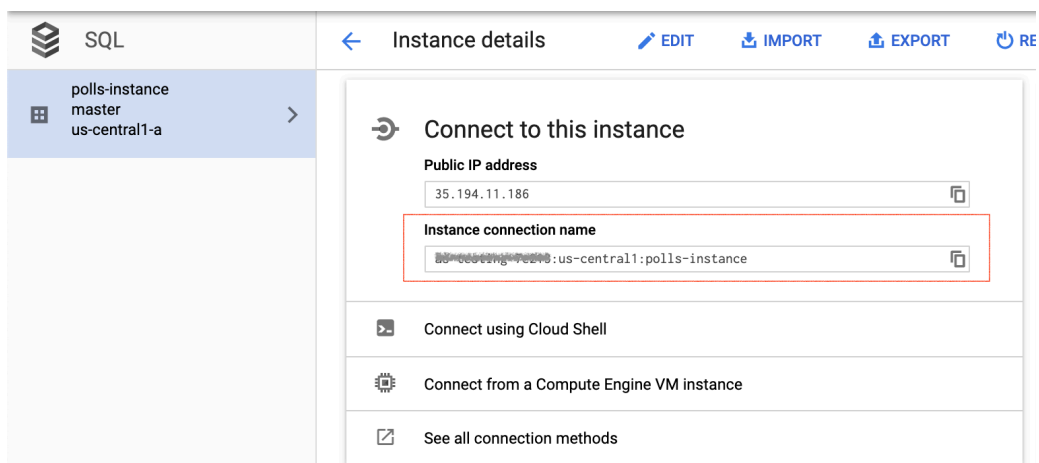
```
chmod +x cloud_sql_proxy
```

建立 Cloud SQL 執行個體

1. 建立 MySQL 適用的 Cloud SQL 第二代執行個體。將執行個體命名為「polls-instance」或類似名稱。執行個體可能需要幾分鐘的時間才能準備就緒。準備就緒後，應該會顯示在執行個體清單中。
2. 使用 gcloud 工具執行下列指令，其中 [YOUR_INSTANCE_NAME] 代表執行個體的名稱。記下系統顯示的執行個體連線名稱值，以供下個步驟使用，格式為 [PROJECT_NAME]:[REGION_NAME]:[INSTANCE_NAME]。

```
gcloud sql instances describe [YOUR_INSTANCE_NAME]
```

或者，您也可以按一下執行個體，查看「執行個體連線名稱」。



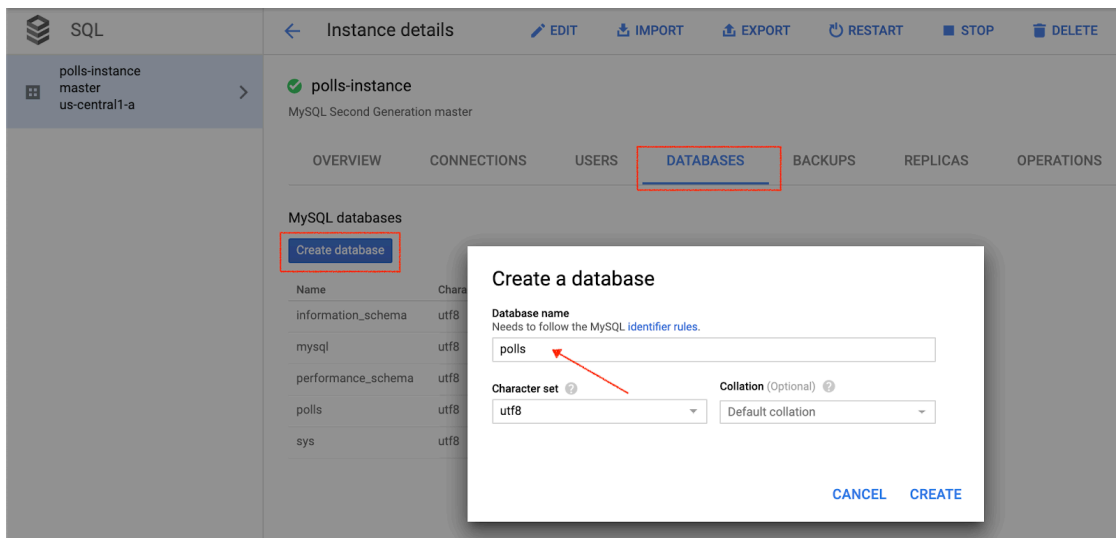
初始化 Cloud SQL 執行個體

使用上一步中的執行個體連線名稱啟動 Cloud SQL Proxy。將 `[YOUR_INSTANCE_CONNECTION_NAME]` 改為您在上一部記下的值。這會將本機電腦連線至執行個體，以利進行本機測試。在本機測試應用程式期間，請讓 Cloud SQL Proxy 保持運作。

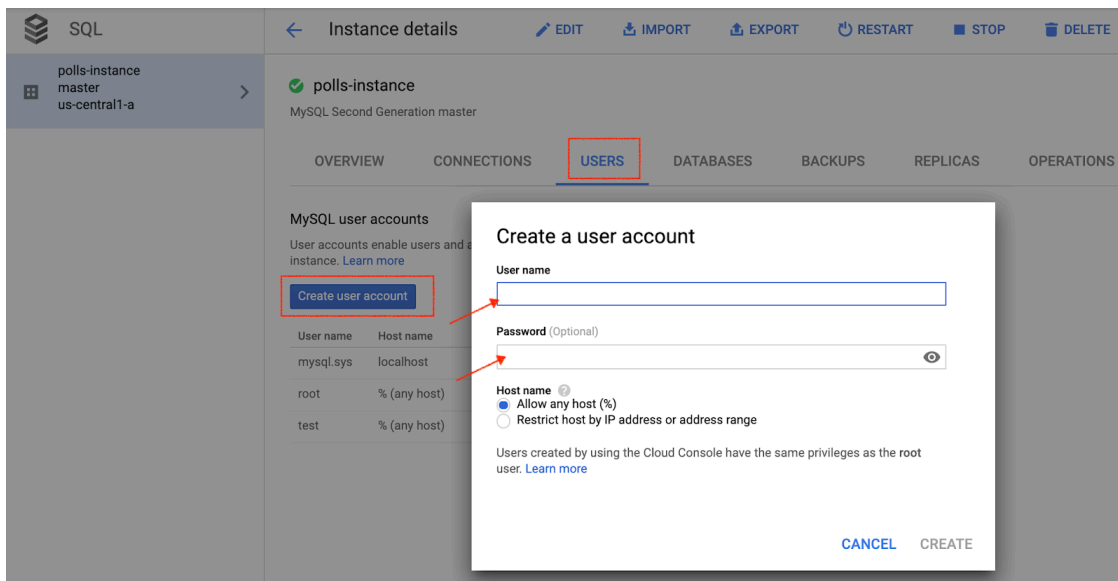
```
./cloud_sql_proxy -instances="[YOUR_INSTANCE_CONNECTION_NAME]"=tcp:3306
```

接著，建立新的 Cloud SQL 使用者與資料庫。

1. 使用 Google Cloud 控制台，為名為 **polls-instance** 的 Cloud SQL 執行個體建立新資料庫。例如，你可以輸入「polls」做為名稱。



2. 使用 Cloud Console 為名為 **polls-instance** 的 Cloud SQL 執行個體建立新使用者帳戶。



進行資料庫設定

1. 開啟 `mysite/settings.py` 進行編輯。
2. 請將 `[YOUR-USERNAME]` 和 `[YOUR-PASSWORD]` 兩處更換為您在上一節中建立的資料庫使用者名稱和密碼。這有助於針對 App Engine 部署及本機測試設定連線至資料庫。

3. 在「[CE_NAME]」 'HOST': 'cloudsql/[PROJECT_NAME]:[REGION_NAME]:[INSTAN ' 這行中，將 [PROJECT_NAME]:[REGION_NAME]:[INSTANCE_NAME] 替換為您在前一節記下的值。
4. 執行下列指令，並複製輸出的執行個體連線名稱值，以供下個步驟使用。

```
gcloud sql instances describe [YOUR_INSTANCE_NAME]
```

5. 將 [YOUR-CONNECTION-NAME] 換成您在前一節記下的值。
6. 將 [YOUR-DATABASE] 替換為您在上一個章節中選擇的名稱。

```
# [START db_setup]
if os.getenv('GAE_APPLICATION', None):
    # Running on production App Engine, so connect to Google Cloud SQL using
    # the unix socket at /cloudsql/<your-cloudsql-connection string>
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.mysql',
            'HOST': '/cloudsql/[PROJECT_NAME]:[REGION_NAME]:[INSTANCE_NAME]',
            'USER': '[YOUR-USERNAME]',
            'PASSWORD': '[YOUR-PASSWORD]',
            'NAME': '[YOUR-DATABASE]',
        }
    }
else:
    # Running locally so connect to either a local MySQL instance or connect to
    # Cloud SQL via the proxy. To start the proxy via command line:
    #     $ cloud_sql_proxy -instances=[INSTANCE_CONNECTION_NAME]=tcp:3306
    # See https://cloud.google.com/sql/docs/mysql-connect-proxy
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.mysql',
            'HOST': '127.0.0.1',
            'PORT': '3306',
            'NAME': '[YOUR-DATABASE]',
            'USER': '[YOUR-USERNAME]',
            'PASSWORD': '[YOUR-PASSWORD]'
        }
    }
# [END db_setup]
```

7. 關閉並儲存 settings.py 。

5. 服務帳戶設定

1. 在 Dialogflow 主控台中，按一下

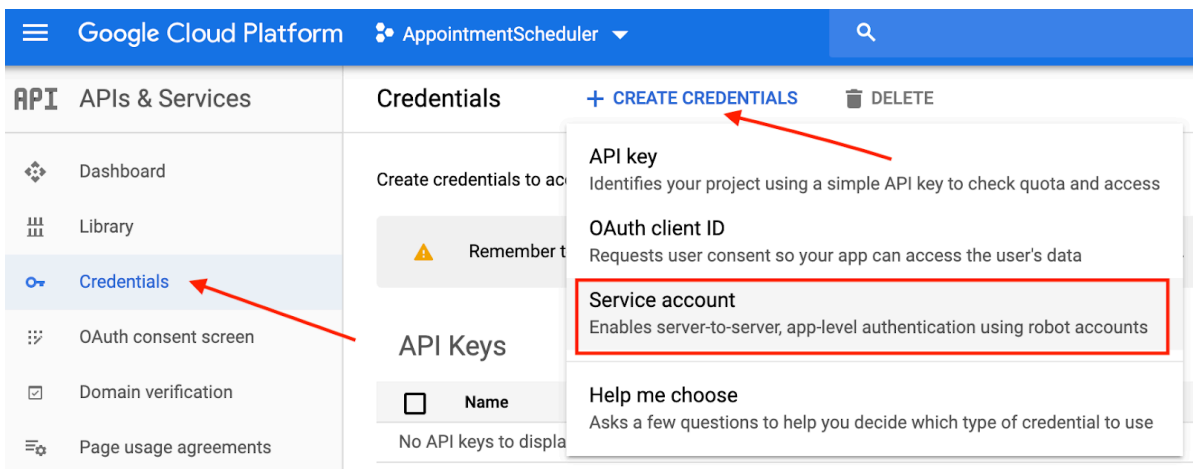


- 在「專案 ID」旁的「一般」分頁中，按一下「Google Cloud」



◦

2. 依序點選「導覽選單」圖示 ≡ > 「API 和服務」 > 「憑證」。
3. 依序按一下「建立憑證」 > 「服務帳戶」。



4. 在「服務帳戶詳細資料」中，輸入「appointment-scheduler」做為「服務帳戶名稱」，然後按一下「建立」。

Create service account

1 Service account details

2 Grant this service account access to project (optional)

Service account details

Service account name

appointment-scheduler

Display name for this service account

Service account ID

appointment-scheduler

@poetic-sentinel-238020.iam.gserviceacc

X

C

Service account description

Describe what this service account will do

CREATE

CANCEL

5. 在「Grant this service account access to project」(將專案存取權授予這個服務帳戶) 頁面，點選「Continue」(繼續)，略過這個步驟。
6. 在「Grant users access to this service account (optional)」(將這個服務帳戶的存取權授予使用者 (選用)) 頁面上，依序點選「Create Key」 > 「JSON」 > 「Create」。

系統會將 JSON 檔案下載至您的電腦，您需要在後續設定章節中使用該檔案。

AppointmentScheduler

Create service account

Service account details

Grant this service account access to project (opt

Grant users access to this service account (optional)

Grant access to users or groups that need to perform actions as this service account.
[Learn more](#)

Service account users role

Grant users the permissions to deploy jobs and VMs with this service account

Service account admins role

Grant users the permission to administer this service account

Create key (optional)

Download a file that contains the private key. Store the file securely because this key can't be recovered if lost. However, if you are unsure why you need a key, skip this step for now.

+ CREATE KEY

DONE

CANCEL

Create key (optional)

Download a file that contains the private key. Store the file securely because this key can't be recovered if lost. However, if you are unsure why you need a key, skip this step for now.

Key type

☒ JSON

Recommended

☐ P12

For backward compatibility with code using the P12 format

CREATE

CANCEL

步驟 6 · 約 1 分鐘

6. 設定要從應用程式呼叫的 Dialogflow detectIntent 端點

1. 在 chat 資料夾中，將 `AppointmentScheduler.json` 換成您的憑證 JSON 檔案。
 2. 在聊天資料夾的 `views.py` 中，將 `GOOGLE_PROJECT_ID = "<YOUR_PROJECT_ID>"` 變更為您的專案 ID。
-

7. 在本機建構及執行應用程式

如要在本機電腦執行 Django 應用程式，就需要設定 Python 開發環境，包括 Python、pip 及 virtualenv。如需操作說明，請參閱「[設定 Python 開發環境](#)」。

1. 建立獨立的 Python 環境，並安裝依附元件：

```
virtualenv env
source env/bin/activate
pip install -r requirements.txt
```

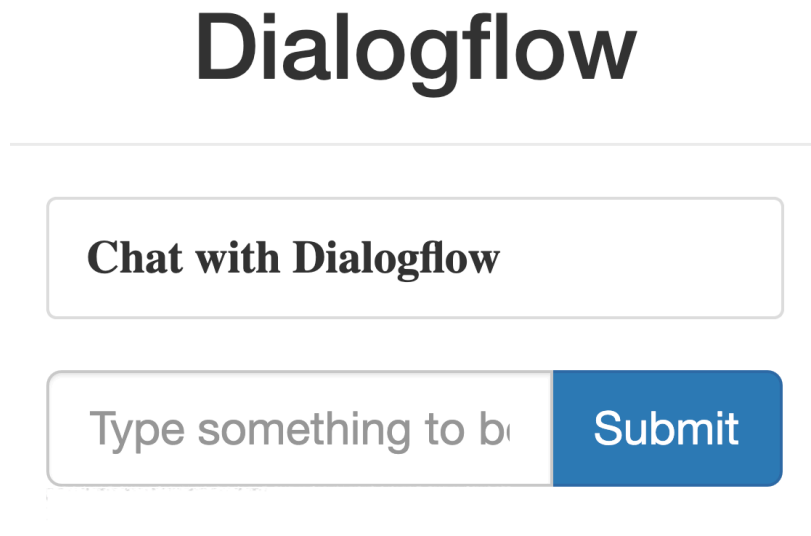
2. 執行 Django 遷移作業以設定模型。

```
python3 manage.py makemigrations
python3 manage.py makemigrations polls
python3 manage.py migrate
```

3. 啟動本機網路伺服器。

```
python3 manage.py runserver
```

4. 在網路瀏覽器中輸入 <http://localhost:8000/>。您應該會看到如下列螢幕截圖所示的簡單網頁：



範例應用程式頁面是由在您電腦上執行的 Django 網路伺服器提供。準備好繼續時，請按下 **Control+S** 鍵 (Macintosh 上的 **Command+S** 鍵) 停止本機網路伺服器。

步驟 8 · 約 3 分鐘

8. 將應用程式部署至 App Engine 標準環境

執行下列指令，將所有應用程式的靜態檔案移至 `settings.py` 中 `STATIC_ROOT` 指定的資料夾：

```
python3 manage.py collectstatic
```

在 `app.yaml` 檔案所在的應用程式目錄中執行下列指令，以上傳應用程式：

```
gcloud app deploy
```

等候更新完成通知訊息。

步驟 9 · 約 1 分鐘

9. 測試前端用戶端

在網路瀏覽器中輸入 `https://<your_project_id>.appspot.com`。

這次您的要求是由在 App Engine 標準環境執行的網路伺服器提供。

`app deploy` 指令會部署 `app.yaml` 中描述的應用程式，並將新部署的版本設為預設版本，將其用來處理所有新流量。

步驟 10 · 約 1 分鐘

10. 正式版

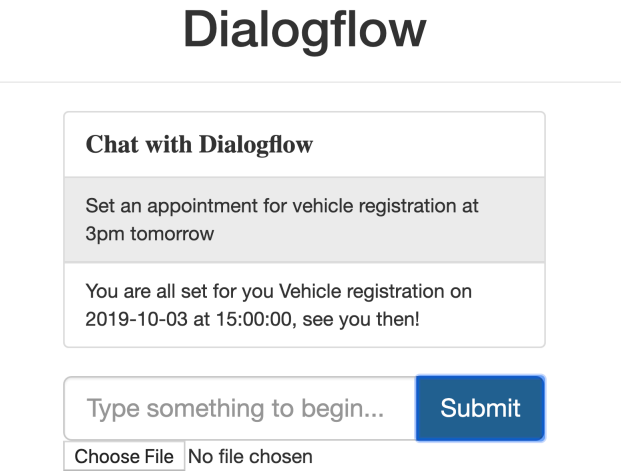
步驟 11 · 約 0 分鐘

11. 準備好在實際工作環境提供內容時，請在 `mysite/settings.py` 中將 `DEBUG` 變數變更為 `False`。

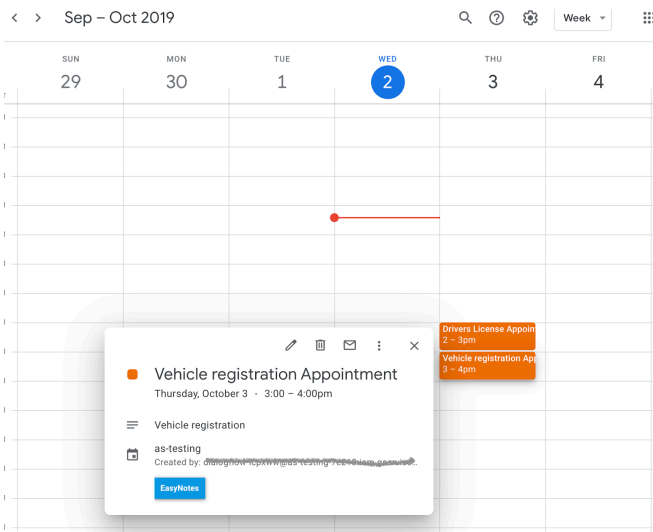
12. 測試您的聊天機器人

前往 https://<your_project_id>.appspot.com，然後輸入下列內容：

- 1. 使用者：「設定明天下午 3 點的車輛登記預約。」
- 2. 聊天機器人會回覆如下：



- 3. 日曆會預訂回覆內容。



13. 清除所用資源

如果你打算完成其他 Dialogflow 程式碼研究室，請先略過這部分，稍後再返回。

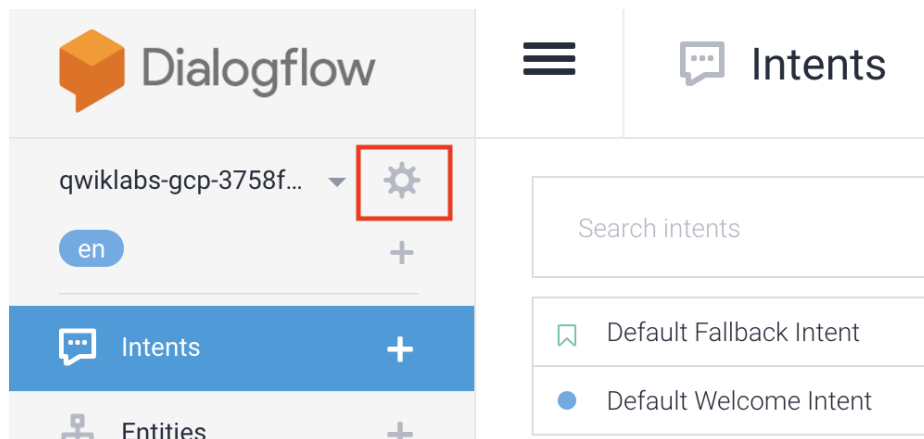
刪除 Dialogflow 代理程式

1. 按一下現有代理程式旁的



。

- 2.



3. 在「一般設定」分頁中，捲動至底部，然後點選「刪除這個代理程式」。
4. 在隨即顯示的視窗中輸入「刪除」，然後按一下「刪除」。

步驟 14 · 約 0 分鐘

14. 恭喜

您已在 Dialogflow 中建立聊天機器人，並與 Google 日曆整合。你現在是聊天機器人開發人員！

瞭解詳情

如要瞭解詳情，請參閱下列資源：

- [將 Vision API 與 Dialogflow 整合](#)
 - [Dialogflow GitHub](#) 頁面上的程式碼範例
-